Gaming Castle

Online Gaming Centre Management System

System Design Specification (SDS)

By

Kirisigan – SE/2021/007

Nirojan – SE/2021/014

Thivya – SE/2021/022

Jashvanth – SE/2021/055

A report submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science Honours in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Version History

Version No.	Date	Author(s)	Summary of Changes
01	2026/04/10	Kirisigan, Nirojan, Thivya, Jashvanth	Initial draft of SDS document
02	2026/04/15	Kirisigan, Jashvanth, Thivya, Nirojan	All diagrams completed, security and deployment sections finalised
03	2026/10/22	Kirisigan, Jashvanth, Thivya, Nirojan	Aligned SDS with the implemented system and SRS v03: six Spring Boot services plus a Socket.IO real-time server, Eureka and Spring Cloud Gateway; isolated database per service on PostgreSQL 16; PS5/PC/VR stations; Saga compensating transactions, Resilience4j circuit breakers, token-bucket rate limiting and in-memory soft slot locks; PayHere IPN and Luhn card validation; reviews; updated class, sequence, UI, security and deployment designs

Table 1: Version History

Abstract

This System Design Specification (SDS) defines the complete technical design for the Gaming Castle Online Gaming Centre Management System — a web-based platform developed to manage PS5, PC Esports, and VR station bookings, tournament registrations, PayHere payment processing, loyalty rewards, session reviews, and administrative operations for a gaming hub located in Nelliyadi, Sri Lanka.

The system is designed using a Microservices-based architecture comprising six independently deployable Spring Boot services (user, booking, payment, tournament, loyalty, notification), a Node.js Socket.IO real-time server, a Spring Cloud API Gateway, and a Eureka service registry.

Each service is internally structured following the Model-View-Controller (MVC) pattern, and the system is deployed on Amazon Web Services (AWS) via Docker containerisation. A strict database-per-service strategy is adopted on PostgreSQL 16, with schemas conforming to the Third Normal Form (3NF) except where documented. Distributed consistency and resilience are achieved with the Saga pattern with compensating transactions, Resilience4j circuit breakers, a token-bucket rate limiter, and a real-time soft lock for slot holds.

Security is enforced through JWT-based authentication, Role-Based Access Control (RBAC), bcrypt password hashing, gateway rate limiting, PayHere IPN signature validation, TLS 1.2/1.3 encryption in transit, and AES-256 encryption at rest. The frontend is built using Angular 19 with standalone components, signals, and OnPush change detection, conforming to WCAG 2.1 Level AA accessibility standards across all device types and verified by 695 automated unit tests. Deployment is automated through a GitHub Actions CI/CD pipeline across Development, Staging, and Production environments.

This document is intended for the development team, academic evaluators, and the client, and is to be read in conjunction with the Software Requirements Specification (SRS) version 03 for the Gaming Castle system.

Table of Contents

Version History ii

Abstract ii

Table of Contents iv

List of Tables vi

List of Figures vii

Chapter 1 Introduction 1

1.1 Purpose 1

1.2 Scope 1

1.3 Definitions, Acronyms and Abbreviations 1

1.4 Document Overview 4

Chapter 2 Architectural Design 4

2.1 High-Level Architecture 4

2.2 Cloud-Native Architecture 6

2.3 Justification for Architecture Choice 8

2.4 Distributed Design Patterns 8

2.5 Scalability and Maintainability Considerations 9

2.6 Technology Stack 10

Chapter 3 Database Design 11

3.1 Overview 11

3.2 Entity-Relationship Diagram 11

3.3 Key Entity Attributes 14

3.4 Normalisation 17

Chapter 4 Class Diagram 17

4.1 Overview 17

4.2 Core Domain Classes 18

4.3 Relationships 22

4.4 Enumerations 23

Chapter 5 Sequence Diagrams 24

5.1 Overview 24

5.2 UC-01: Register Account 25

5.3 UC-02: Login to System 26

5.4 UC-03: Book Gaming Slot 28

5.5 UC-04: Cancel / Reschedule Booking **30**
5.6 UC-05: Admin Book Slot for Walk-in Customer **31**
5.7 UC-06: Make Online Payment (PayHere, IPN and Saga) **33**
5.8 UC-08: Register for Tournament **34**
5.9 UC-11: Redeem Loyalty Points **36**
5.10 UC-15: Submit Session Review **38**
Chapter 6 UI/UX Design **39**
6.1 Overview **39**
6.2 Design Principles **39**
6.3 Key Screens and Wireframe Descriptions **39**
6.3.1 Customer – Home Page **39**
6.3.2 Customer – Slot Booking Page **40**
6.3.3 Customer – Payment Page **42**
6.3.4 Customer – Tournament Page **42**
6.3.5 Customer – My Bookings and My Reviews **43**
6.3.6 Admin – Dashboard **44**
6.3.7 Admin – Booking Management **45**
6.4 Navigation Flow **47**
6.5 Accessibility Considerations **47**
Chapter 7 Security Design **48**
7.1 Authentication Mechanism **48**
7.2 Authorisation Levels **49**
7.3 Data Protection Approach **50**
7.4 Input Validation and Sanitisation Strategy **51**
7.5 Rate Limiting and Abuse Protection **51**
7.6 Encryption Standards **51**
7.7 Account Security **52**
Chapter 8 Deployment Design **53**
8.1 Hosting Plan **53**
8.2 Hardware Requirements **54**
8.3 Environment Specification **54**
8.4 CI/CD Pipeline Overview **55**
8.5 Network Design **56**
Chapter 9 References **57**

List of Tables

Table 1: Version History
Table 2: Definitions, Acronyms and Abbreviations
Table 3: Microservices
Table 4: Architecture Comparison
Table 5: Technology Stack
Table 6: Databases per Service
Table 7: Entity-Relationship Summary
Table 8: Key Entity Attributes
Table 9: Core Domain Classes
Table 10: Domain Enumerations
Table 11: Sequence – UC-01 Register Account
Table 12: Sequence – UC-02 Login to System
Table 13: Sequence – UC-03 Book Gaming Slot
Table 14: Sequence – UC-04 Cancel / Reschedule Booking
Table 15: Sequence – UC-05 Admin Walk-in Booking
Table 16: Sequence – UC-06 Online Payment
Table 17: Sequence – UC-08 Register for Tournament
Table 18: Sequence – UC-11 Redeem Loyalty Points
Table 19: Sequence – UC-15 Submit Session Review
Table 20: Navigation Flow by Role
Table 21: Role-Based Access Control Matrix
Table 22: Encryption Standards
Table 23: Hosting Plan
Table 24: Environment Specification
Table 25: CI/CD Pipeline Stages

List of Figures

Figure 1: High-Level Architecture Diagram
Figure 2: Cloud-Native Architecture
Figure 3: Entity-Relationship Diagram
Figure 4: Class Diagram
Figure 5: Sequence – UC-01 Register Account
Figure 6: Sequence – UC-02 Login to System
Figure 7: Sequence – UC-03 Book Gaming Slot
Figure 8: Sequence – UC-05 Admin Walk-in Booking
Figure 9: Sequence – UC-08 Register for Tournament
Figure 10: Sequence – UC-11 Redeem Loyalty Points
Figure 11: UI/UX Design - Customer – Home Page
Figure 12: UI/UX Design - Customer – Slot Booking
Page Figure 13: UI/UX Design - Customer – Payment
Page Figure 14: UI/UX Design - Customer – Tournament
Page Figure 15: UI/UX Design - Admin – Dashboard
Figure 16: UI/UX Design - Admin – Booking Management
Figure 17: CI/CD Pipeline Diagram
Figure 18: Network Architecture Diagram

Chapter 1 Introduction

1.1 Purpose

This System Design Specification (SDS) document defines the technical design decisions for the Gaming Castle Online Gaming Centre Management System. This document serves as the primary technical reference for the development team and describes the system architecture, database design, class structure, sequence diagrams, UI/UX design, security design, and deployment strategy.

The intended audience of this document includes the development team, academic evaluators, and the client. This document is to be read in conjunction with the Software Requirements Specification (SRS) for Gaming Castle.

1.2 Scope

The Gaming Castle system is a web-based platform designed to manage slot bookings for PS5 console stations, PC Esports rigs, and VR stations, together with tournament registrations, payment processing, loyalty rewards, session reviews, real-time notifications, and administrative operations for a gaming hub in Nelliyadi, Sri Lanka. This SDS document provides the complete technical design required to implement the system as defined in the SRS.

1.3 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
SDS	System Design Specification
SRS	Software Requirements Specification
MVC	Model-View-Controller architectural pattern
API	Application Programming Interface

Term / Acronym	Definition
ER Diagram	Entity-Relationship Diagram
JWT	JSON Web Token
RBAC	Role-Based Access Control
CI/CD	Continuous Integration / Continuous Deployment
WCAG	Web Content Accessibility Guidelines
TLS	Transport Layer Security
VPC	Virtual Private Cloud
CDN	Content Delivery Network
SPA	Single Page Application
Station	A bookable gaming unit of type CONSOLE (PS5), PC, or VR
Soft Lock	A temporary 5-minute in-memory reservation of a slot during checkout

TTL	Time To Live

Term / Acronym	Definition
Saga	A sequence of local transactions across services with compensating actions on failure
Compensating Transaction	A reverse action that undoes a completed step of a Saga
Circuit Breaker	A resilience pattern that stops calls to a failing dependency and returns a fallback
Token Bucket	A rate-limiting algorithm that allows bursts up to a capacity and refills at a fixed rate
IPN	PayHere Instant Payment Notification webhook
Luhn algorithm	Mod-10 checksum for validating card numbers
Eureka	Netflix service registry used for service discovery
Feign	Declarative HTTP client used for inter-service REST calls
Socket.IO	Library for real-time bidirectional events over WebSocket
LKR	Sri Lankan Rupee

Table 2: Definitions, Acronyms and Abbreviations

1.4 Document Overview

This document is organised as follows:

- Chapter 1 – Introduction: Provides the purpose, scope, definitions, and overview.
- Chapter 2 – Architectural Design: Describes the high-level architecture, distributed patterns, and technology stack.
- Chapter 3 – Database Design: Presents the per-service schemas and normalisation.
- Chapter 4 – Class Diagram: Details classes, attributes, methods, and relationships.
- Chapter 5 – Sequence Diagrams: Illustrates interaction flows for the major use cases.
- Chapter 6 – UI/UX Design: Covers wireframes, user flow, and accessibility.
- Chapter 7 – Security Design: Defines authentication, authorisation, and data protection.
- Chapter 8 – Deployment Design: Specifies the hosting plan, CI/CD pipeline, and environments.
- Chapter 9 – References: Lists all cited sources and standards.

Chapter 2 Architectural Design

2.1 High-Level Architecture

The Gaming Castle system shall be implemented using a Microservices-based Architecture with a layered internal structure within each service. The system comprises the following tiers:

- Client Tier: Angular 19 single-page application using standalone components, reactive signals, OnPush change detection, and a custom SCSS design system, conforming to WCAG 2.1 AA.
- Real-Time Event Tier: A Node.js Socket.IO server managing live rooms, in-memory TTL slot locks, and an HTTP event bridge that backend services call to publish events.
- API Gateway Tier: Spring Cloud Gateway (port 8080) for routing, JWT validation and forwarding, CORS, and the AuthRateLimitFilter.
- Discovery Tier: A Eureka server (port 8761) providing registration and heartbeat-based discovery.

- Microservice Domain Tier (Spring Boot 3, Java 17): six independently deployable services, each responsible for a distinct business domain.

The six core microservices are:

Service	Port	Responsibility
user-service	8081	Registration, login (email or phone), OTP password reset, deactivation and reactivation, profiles and avatars, admin account management
booking-service	8082	Stations, games, availability, bookings (online and walk-in), cancellation and rescheduling, reviews, Saga orchestration
payment-service	8083	PayHere adapter, MD5 signatures, IPN webhook, cash payments, receipts, payment history
tournament-service	8084	Tournaments, brackets, registrations, ticker data
loyalty-service	8085	Point accrual, redemption, reversal, tiers, audit log
notification-service	8086	Email (JavaMail SMTP) and SMS (Twilio) delivery

Table 3: Microservices

Each microservice internally follows the Model-View-Controller (MVC) layered pattern with a defined separation of concerns across the Controller layer (REST API endpoints), Service layer (business logic), Repository layer (database access), and Model layer (data entities). The earlier design listed five services; the notification service is now a separate sixth service.

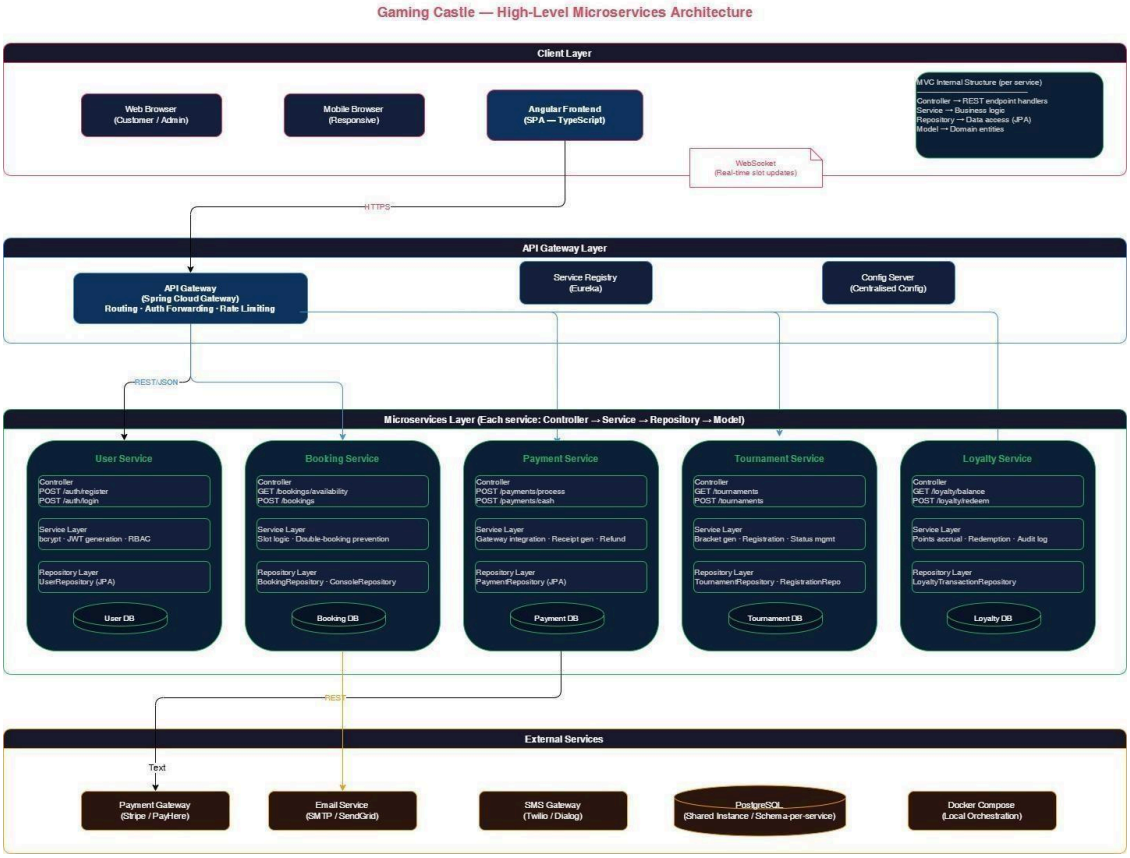


Figure 1: High-Level Architecture Diagram

Note: diagram shown is from version 02 and is pending update to match version 03.

2.2 Cloud-Native Architecture

The system is designed as a cloud-native application to ensure scalability, resilience, and ease of deployment. The following cloud-native design principles are applied:

- Containerisation: Every service, the real-time server, the gateway, and the registry shall be packaged as Docker containers, ensuring environment consistency across development, staging, and production.
- API Gateway: Spring Cloud Gateway shall serve as the single entry point for all client requests, handling routing, authentication forwarding, CORS, and rate limiting.
- Service Registry: Eureka shall be used for microservice discovery, enabling dynamic routing and fault tolerance.
- Configuration: Environment-specific configuration shall be supplied through environment variables and deployment definitions (Docker Compose locally,

ECS task definitions in AWS); a dedicated configuration server may be added if centralised configuration is required.

- Horizontal Scaling: Stateless services may be scaled independently by adding container instances behind a load balancer, supporting the NFR requirement of a 3x user load increase without redesign. The real-time server holds slot locks in memory and therefore runs as a single instance unless a shared store is introduced (see 2.5).

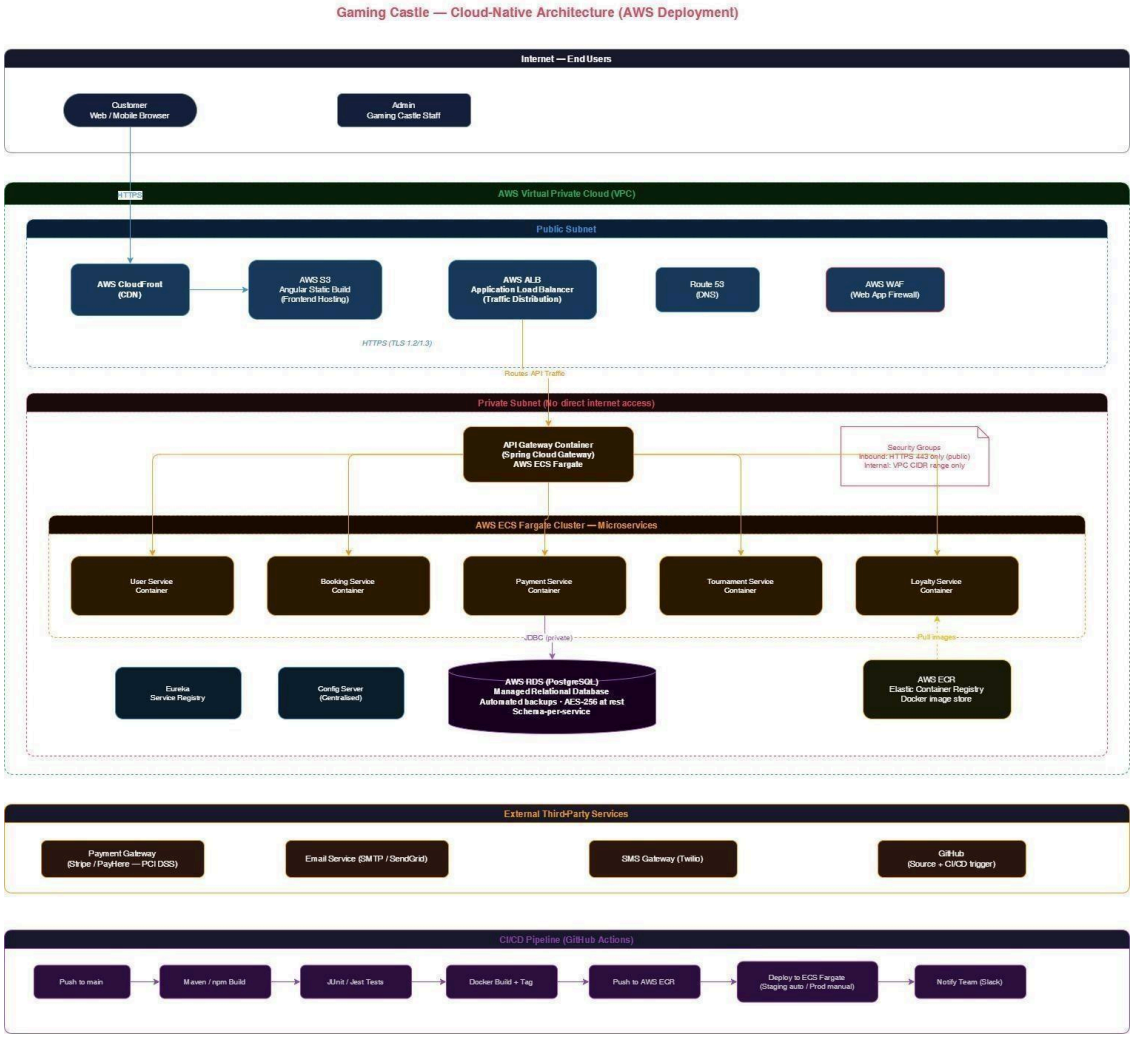


Figure 2: Cloud-Native Architecture

Note: diagram shown is from version 02 and is pending update to match version 03.

2.3 Justification for Architecture Choice

The Microservices architecture was selected over monolithic and layered alternatives for the following reasons:

Criterion	Microservices (Selected)	Monolithic (Rejected)
Independent Deployment	Each service deployed independently	Full redeployment required
Scalability	Per-service horizontal scaling	Scale entire application
Fault Isolation	Failure in one service does not cascade (circuit breakers)	Single point of failure
Team Alignment	Each member owns a domain service	Shared codebase conflicts
Maintainability	Small, focused, testable services	Large codebase increases complexity

Table 4: Architecture Comparison

2.4 Distributed Design Patterns

1. Saga pattern with compensating transactions. Because each service owns its database, a booking that spans booking, payment, and loyalty cannot use one ACID transaction. The booking service orchestrates the flow and records compensations in the `compensating_transactions` table (booking_id, transaction_type, status PENDING/COMPLETED/FAILED, payload JSONB, retry_count). On payment failure or cancellation, `SagaCompensationService` executes reverse actions: restoring or reversing loyalty points and releasing slot holds, retrying failed compensations. This gives eventual consistency (NFR-20).

2. Circuit breaker (Resilience4j). `PaymentClient` and `LoyaltyClient` are annotated with `@CircuitBreaker` (for example, name `paymentService` with a fallback method such

as `paymentFallback`). After consecutive failures the circuit opens and a fallback is returned, preventing thread exhaustion and cascading failure (NFR-19).

3. **Token-bucket rate limiting.** The gateway's `AuthRateLimitFilter` (filter order -2) keeps an in-memory bucket per client IP with capacity 10 and a refill of 5 tokens per second. When the bucket is empty the gateway returns HTTP 429 with a `Retry-After` header (NFR-21).

4. **Soft-lock sidecar.** The Socket.IO server acts as a low-latency stateful lock manager so that short-lived holds do not create write contention in the relational database. The database unique constraint on the booking slot remains the authoritative guard against double-booking.

5. **Database per service.** Each service owns an isolated PostgreSQL 16 database. No cross-database SQL is used; services communicate through REST (Feign clients) and Socket.IO events, and refer to other services' data by identifier only.

2.5 Scalability and Maintainability Considerations

- Each microservice may be independently upgraded, replaced, or scaled without affecting others.
- Docker Compose shall be used for local development orchestration; AWS ECS Fargate is used for production deployment.
- A strict database-per-service pattern shall be adopted, ensuring loose coupling between data stores.
- All inter-service communication shall be conducted via RESTful HTTP APIs using JSON payloads, protected by circuit breakers.
- WebSocket connections (Socket.IO) shall be used for real-time slot lock, unlock, booked, notification, and tournament updates, published by backend services through the HTTP event bridge.
- Known limitation: in-memory locks are lost if the real-time server restarts and cannot be shared across multiple instances. Locks expire within 5 minutes, clients resynchronise on reconnect, and the database constraint prevents double-booking.

A shared store (for example, Redis with the Socket.IO adapter) is the recommended path for multi-instance scaling.

2.6 Technology Stack

Layer	Technology
Frontend	Angular 19 (TypeScript, standalone components, signals, OnPush), SCSS design system
Real-time	Node.js with Socket.IO
Backend	Spring Boot 3, Java 17, Spring Cloud Gateway, Eureka, OpenFeign, Resilience4j
Database	PostgreSQL 16 (one database per service)
Authentication	JWT, bcrypt
Payments	PayHere (LKR) with MD5 signature and IPN webhook
Notifications	JavaMail (SMTP), Twilio (SMS)
Testing	JUnit, Mockito, @WebMvcTest, @DataJpaTest; 695 frontend unit tests in headless Chrome
Containerisation	Docker, Docker Compose
CI/CD	GitHub Actions
Cloud	AWS (ECS Fargate, RDS, S3, CloudFront, ALB, ECR)

Table 5: Technology Stack

Chapter 3 Database Design

3.1 Overview

PostgreSQL 16 shall be used as the primary data store for the Gaming Castle system. A strict database-per-service strategy is adopted for the microservices architecture: each service owns a separate database, with no cross-database queries or foreign keys. Relationships between services are logical references by UUID, resolved through REST calls.

Service	Database
user-service	gamingcastle_users
booking-service	gamingcastle_bookings
payment-service	gamingcastle_payments
tournament-service	gamingcastle_tournaments
loyalty-service	gamingcastle_loyality (name as configured in the deployment)
notification-service	gamingcastle_notifications

Table 6: Databases per Service

3.2 Entity-Relationship Diagram

The following key entities and relationships constitute the data model for the Gaming Castle system, grouped by owning service. Links across services (marked "logical") are identifiers only.

Entity	Service	Primary Key	Foreign Keys / Relationships
User	user	id	None (root entity)
VerificationCode	user	id	user_id → User
GameStation	booking	id	None
Game	booking	id	None
StationGame	booking	station_id, game_id	station_id → GameStation, game_id → Game
Booking	booking	id	station_id → GameStation, game_id → Game; user_id → User (logical)
Review	booking	id	booking_id → Booking (unique); user_id → User (logical), station_id → GameStation
CompensatingTransaction	booking	id	booking_id → Booking
Payment	payment	id	booking_id → Booking (logical), user_id → User (logical)

Entity	Service	Primary Key	Foreign Keys / Relationships
LoyaltyAccount	loyalty	id	user_id → User (unique, logical)
LoyaltyTransaction	loyalty	id	account_id → LoyaltyAccount; booking_id → Booking (logical)
Tournament	tournament	id	game_id → Game (logical)
TournamentRegistration	tournament	id	tournament_id → Tournament; user_id → User, payment_id → Payment (logical)

Table 7: Entity-Relationship Summary

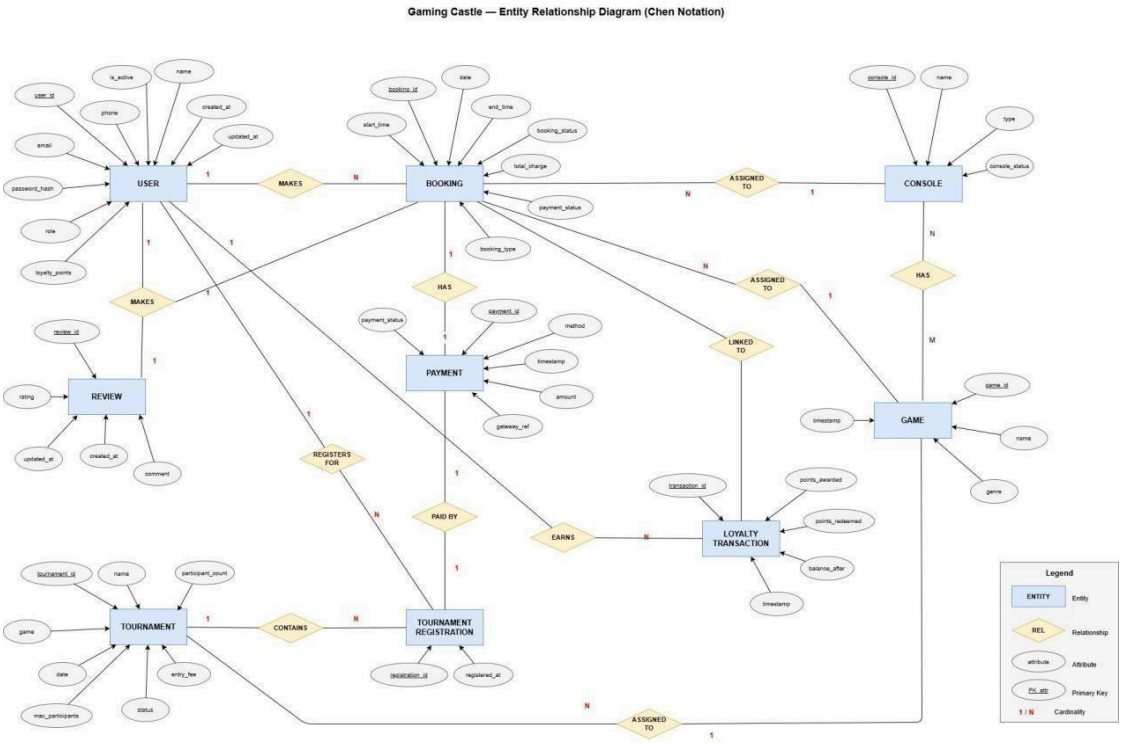


Figure 3: Entity-Relationship Diagram

Note: diagram shown is from version 02 and is pending update to match version 03.

3.3 Key Entity Attributes

Entity	Attributes	Notes
game_stations (booking)	id (UUID), station_code, name, type (CONSOLE, PC, VR), hourly_rate (NUMERIC), active, image_url, created_at	Hourly rate stored per station
games (booking)	id (UUID), title, genre, cover_url, platforms (TEXT[]), active, featured	Game catalogue
station_games (booking)	station_id (FK), game_id (FK)	Many-to-many mapping

Entity	Attributes	Notes
bookings (booking)	id (UUID), booking_reference, user_id, station_id (FK), game_id (FK), booking_date, start_time (TIMESTAMPTZ), end_time (TIMESTAMPTZ), total_amount, discount_amount, final_amount, status (PENDING, CONFIRMED, CANCELLED, COMPLETED), booking_type (ONLINE, WALKIN)	Double-booking prevented by unique constraint
reviews (booking)	id (UUID), booking_id (UUID, UNIQUE), user_id, station_id, rating (1–5), comment (TEXT), created_at, updated_at	One review per completed booking
compensating_transactions (booking)	id (UUID), booking_id, transaction_type, status (PENDING, COMPLETED, FAILED), payload (JSONB), retry_count (INT)	Saga audit and retry
payments (payment)	id (UUID), booking_id, user_id, amount, discount_amount, method (PAYHERE, CARD, CASH), status (PENDING, COMPLETED, FAILED, REFUNDED), transaction_reference, gateway_order_id, payhere_payment_id, loyalty_points_used (INT)	No card data stored; gateway ids null for cash

Entity	Attributes	Notes
users (user)	id (UUID), name, email (UNIQUE), phone (UNIQUE), password_hash, role (CUSTOMER, ADMIN), avatar_url, is_active, failed_login_attempts (INT), lockout_until (TIMESTAMPTZ)	bcrypt hashed password; lockout after repeated failures
verification_codes (user)	id, user_id, code, purpose (PASSWORD_RESET, REACTIVATION), expires_at, used	Time-limited OTP codes
loyalty_accounts (loyalty)	id (UUID), user_id (UNIQUE), balance_points (INT), tier (BRONZE, SILVER, GOLD)	Balance moved out of the user table
loyalty_transactions (loyalty)	id (UUID), account_id (FK), booking_id, transaction_type (EARNED, REDEEMED, REVERSED), points (INT), balance_after (INT), created_at	Full audit trail of points history
tournaments (tournament)	id (UUID), name, game_id, date, entry_fee, max_participants, participant_count, status, image_url, reminder_sent (BOOLEAN)	Auto-close s registration when full
tournament_registrations (tournament)	id (UUID), tournament_id (FK), user_id, user_name, user_email, payment_id, registered_at	Unique (tournament_id, user_id)

Table 8: Key Entity Attributes

3.4 Normalisation

Each service schema is designed to conform to the Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity. The following principles are applied:

- First Normal Form (1NF): All attributes contain atomic values. No repeating groups are used. Each table has a primary key.
- Second Normal Form (2NF): All non-key attributes are fully functionally dependent on the entire primary key. No partial dependencies exist in any table.
- Third Normal Form (3NF): All non-key attributes depend directly on the primary key and not on other non-key attributes. Transitive dependencies are eliminated. For example, customer name and email are stored in the users table and referenced elsewhere by user_id.
- Indexes: Indexes shall be applied on frequently queried foreign keys (user_id, station_id, booking_id) and on date/time columns in the bookings table to support performant calendar queries.

Two deliberate, documented denormalisations result from the database-per-service strategy:

- tournament_registrations stores user_name and user_email as a snapshot of the registrant, avoiding a cross-service call when listing participants or sending cancellation notices.
- reviews.station_id is derivable from the booking but is stored to support station rating aggregation without a join to bookings.

Chapter 4 Class Diagram

4.1 Overview

The class diagram below defines the main domain model classes for the Gaming Castle system, grouped by owning service. The classes correspond to the data entities and service-layer components identified in the architectural and database design. Each class includes its attributes (with visibility and type), methods, and relationships. Classes in different services refer to each other by identifier only.

4.2 Core Domain Classes

Class	Key Attributes	Key Methods
User	- userId: UUID - name: String - email: String - phone: String - passwordHash: String - role: Role - avatarUrl: String - isActive: boolean - failedLoginAttempts: int - lockoutUntil: DateTime	+ register(): void + login(emailOrPhone, password): Token + requestPasswordReset(channel): void + resetPassword(code, newPwd): void + updateProfile(details): void + deactivate(): void + reactivate(code): void
VerificationCode	- id: UUID - userId: UUID - code: String - purpose: VerificationPurpose - expiresAt: DateTime - used: boolean	+ isValid(): boolean + markUsed(): void
GameStation	- stationId: UUID - stationCode: String - name: String - type: StationType - hourlyRate: BigDecimal - active: boolean - imageUrl: String - games: List<Game>	+ getAvailableSlots(date): List<TimeSlot> + getAverageRating(): double + activate(): void + deactivate(): void
Game	- gameId: UUID - title: String - genre: String - coverUrl: String - platforms: List<String> - active: boolean - featured: boolean	+ create(): void + update(): void + deactivate(): void + getByStation(stationId): List<Game>

Class	Key Attributes	Key Methods
Booking	- bookingId: UUID - bookingReference: String - userId: UUID - stationId: UUID - gameId: UUID - bookingDate: LocalDate - startTime: DateTime - endTime: DateTime - totalAmount: BigDecimal - discountAmount: BigDecimal - finalAmount: BigDecimal - status: BookingStatus - bookingType: BookingType	+ create(details): Booking + confirm(): void + cancel(): void (rejected with TOO_LATE within 2 hours) + reschedule(newSlot): void + calculateCharge(): BigDecimal + complete(): void
Review	- reviewId: UUID - bookingId: UUID - userId: UUID - stationId: UUID - rating: int (1–5) - comment: String - createdAt: DateTime - updatedAt: DateTime	+ addReview(bookingId, userId, rating, comment): Review + updateReview(reviewId, rating, comment): void + getReviewByBooking(bookingId) : Review + getUserReviews(userId): List<Review> + getStationAverage(stationId): double

CompensatingTransaction	- id: UUID - bookingId: UUID - transactionType: String - status: CompensationStatus - payload: JSON - retryCount: int	+ execute(): void + retry(): void + markCompleted(): void

Class	Key Attributes	Key Methods
Payment	- paymentId: UUID - bookingId: UUID - userId: UUID - amount: BigDecimal - discountAmount: BigDecimal - method: PaymentMethod - status: PaymentStatus - transactionReference: String - gatewayOrderId: String - payherePaymentId: String - loyaltyPointsUsed: int	+ createPayHereRequest(): Signature + handleIpn(notification): void + recordCashPayment(): void + generateReceipt(): Receipt + refund(): void
LoyaltyAccount	- accountId: UUID - userId: UUID - balancePoints: int - tier: LoyaltyTier	+ getBalance(userId): int + award(booking): void + redeem(points, bookingId): void + reverse(bookingId): void

LoyaltyTransaction	- transactionId: UUID - accountId: UUID - bookingId: UUID - type: LoyaltyTransactionType - points: int - balanceAfter: int - createdAt: DateTime	+ getHistory(userId): List<LoyaltyTransaction>

Class	Key Attributes	Key Methods
Tournament	- tournamentId: UUID - name: String - gameId: UUID - date: LocalDate - entryFee: BigDecimal - maxParticipants: int - participantCount: int - status: TournamentStatus - imageUrl: String - reminderSent: boolean	+ create(details): Tournament + update(details): void + cancel(): void + updateStatus(status): void + generateBracket(): Bracket + isRegistrationOpen(): boolean
TournamentRegistration	- registrationId: UUID - tournamentId: UUID - userId: UUID - userName: String - userEmail: String - paymentId: UUID - registeredAt: DateTime	+ register(user, tournament): void + cancelRegistration(): void + notifyParticipant(): void
SlotLock (real-time server, in-memory)	- stationId: UUID - slotStart: DateTime - lockedBy: UUID - expiresAt: DateTime (TTL 300 s)	+ lock(slot, user): boolean + unlock(slot): void + expire(): void + broadcast(event): void

Table 9: Core Domain Classes

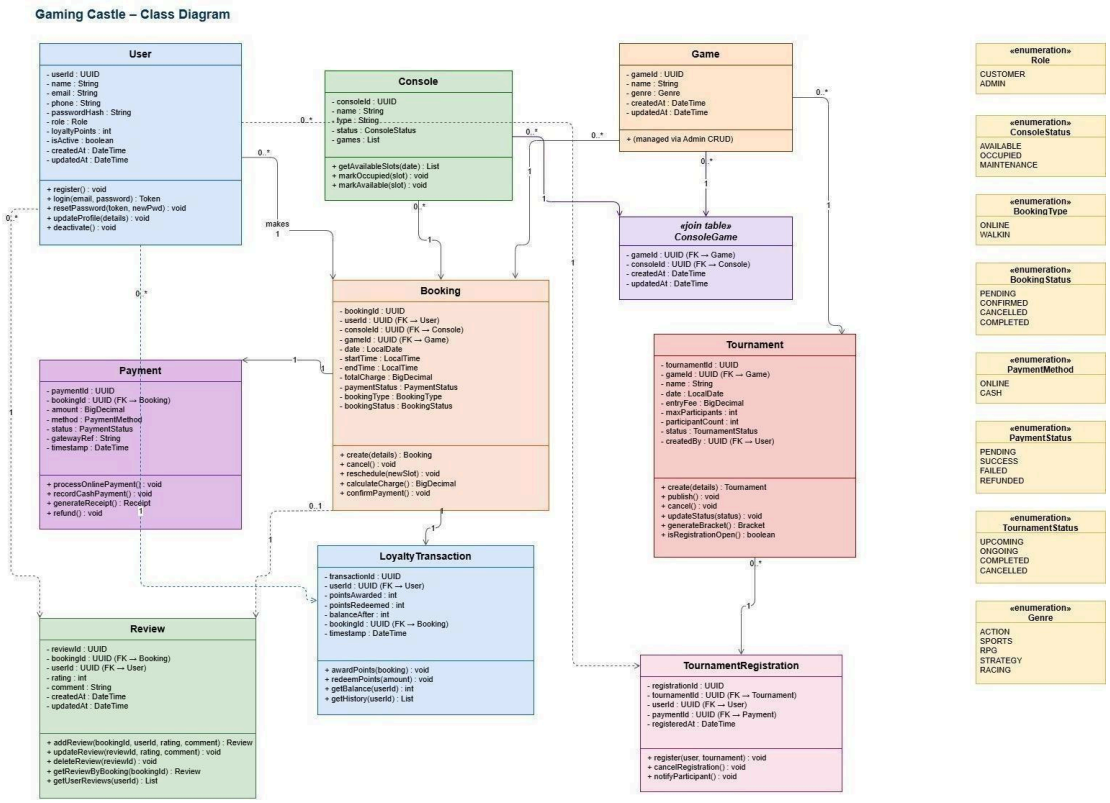


Figure 4: Class Diagram

Note: diagram shown is from version 02 and is pending update to match version 03.

4.3 Relationships

- User – Booking: One-to-Many (logical, across services). A single user may hold multiple bookings.
- GameStation – Booking: One-to-Many. A single station may be associated with multiple bookings across different time slots.
- GameStation – Game: Many-to-Many through station_games. A station can support multiple games, and a game can be available on multiple stations.
- Game – Booking: One-to-Many. A single game may be associated with multiple bookings, while each booking is for one selected game compatible with its station.
- Booking – Review: One-to-One. Each booking can have at most one review.
- User – Review: One-to-Many. A user may write multiple reviews for different bookings.
- GameStation – Review: One-to-Many. Reviews are aggregated per station.
- Booking – Payment: One-to-One (logical, across services). Each booking is associated with one payment record.

- Booking – CompensatingTransaction: One-to-Many. A booking may require several compensating actions.
- User – LoyaltyAccount: One-to-One (logical).
- LoyaltyAccount – LoyaltyTransaction: One-to-Many. Each EARNED, REDEEMED, or REVERSED event is recorded.
- Booking – LoyaltyTransaction: One-to-Many. A booking may earn, redeem, and later reverse points.
- User – TournamentRegistration: One-to-Many. A user may register for multiple tournaments.
- Tournament – TournamentRegistration: One-to-Many. A tournament may have multiple registered participants.
- Game – Tournament: One-to-Many (logical). A single game can have multiple tournaments, while each tournament is associated with one game.

4.4 Enumerations

Enumeration	Values
Role	CUSTOMER, ADMIN
StationType	CONSOLE, PC, VR
BookingType	ONLINE, WALKIN
BookingStatus	PENDING, CONFIRMED, CANCELLED, COMPLETED
PaymentMethod	PAYHERE, CARD, CASH
PaymentStatus	PENDING, COMPLETED, FAILED, REFUNDED

Enumeration	Values
LoyaltyTransactionType	EARNED, REDEEMED, REVERSED
LoyaltyTier	BRONZE, SILVER, GOLD
CompensationStatus	PENDING, COMPLETED, FAILED
VerificationPurpose	PASSWORD_RESET, REACTIVATION
TournamentStatus	UPCOMING, ONGOING, COMPLETED, CANCELLED

Table 10: Domain Enumerations

Changes from version 02: Console is replaced by GameStation with StationType; the ConsoleStatus enumeration is removed (occupancy is derived from bookings and live locks); PaymentMethod ONLINE becomes PAYHERE and CARD; PaymentStatus SUCCESS becomes COMPLETED; and the Genre enumeration becomes a free-text genre on Game.

Chapter 5 Sequence Diagrams

5.1 Overview

This chapter presents sequence diagrams for all major use cases of the Gaming Castle system. Each diagram describes the temporal interaction between actors and system components, including the frontend (Angular), Socket.IO real-time server, API Gateway, relevant microservice, and the database.

5.2 UC-01: Register Account

The registration sequence involves client-side input submission, gateway rate limiting, server-side field validation, uniqueness checking, password hashing, account creation, loyalty account initialisation, and confirmation email dispatch.

Step	From	To	Message / Action
1	Customer (Browser)	Angular Frontend	Submit registration form (name, email, phone, password)
2	Angular Frontend	API Gateway	POST /api/auth/register
3	API Gateway	API Gateway	Apply rate limit (AuthRateLimitFilter); HTTP 429 if exceeded
4	API Gateway	User Service	Forward registration request
5	User Service	User DB	SELECT: check email and phone uniqueness
6	User Service	User Service	Hash password using bcrypt
7	User Service	User DB	INSERT: new user, role=CUSTOMER
8	User Service	Loyalty Service	Create loyalty account with 0 points (REST, circuit breaker)
9	User Service	Notification Service	Send confirmation email

Step	From	To	Message / Action
10	User Service	Angular Frontend	201 Created – Redirect to Login page

Table 11: Sequence – UC-01 Register Account

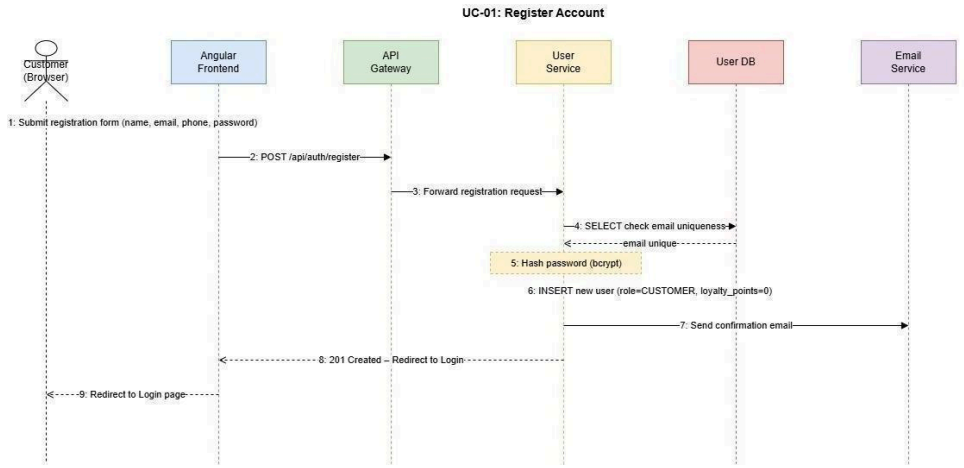


Figure 5: Sequence – UC-01 Register Account

Note: diagram shown is from version 02 and is pending update to match version 03.

5.3 UC-02: Login to System

Step	From	To	Message / Action
1	User (Browser)	Angular Frontend	Submit login form (email or phone, password)
2	Angular Frontend	API Gateway	POST /api/auth/login or /api/auth/login/phone
3	API Gateway	API Gateway	Token-bucket check (5 req/s, burst 10); HTTP 429 with Retry-After if exceeded

Step	From	To	Message / Action
4	API Gateway	User Service	Forward login request
5	User Service	User DB	SELECT user; verify active and not locked; validate bcrypt hash
6	User Service	User DB	On failure increment failed_login_attempts; set lockout_until at 5 failures
7	User Service	User Service	Determine role; generate access and refresh JWTs
8	User Service	Angular Frontend	200 OK – tokens (HTTP-only cookies) and role
9	Angular Frontend	Browser	Redirect to Customer or Admin Dashboard; connect to Socket.IO and join the user's room

Table 12: Sequence – UC-02 Login to System

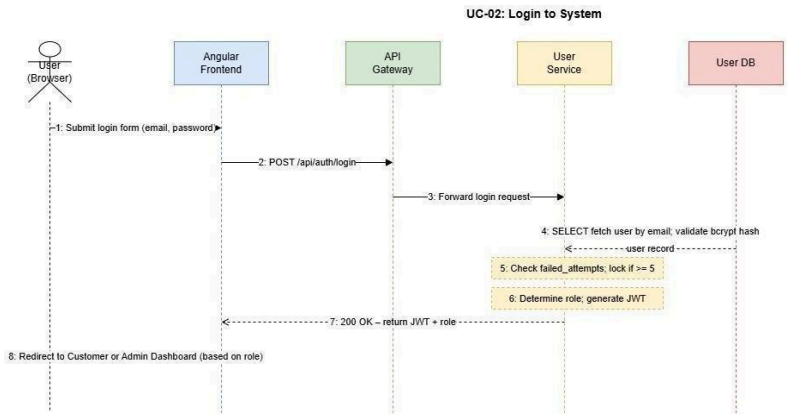


Figure 6: Sequence – UC-02 Login to System

Note: diagram shown is from version 02 and is pending update to match version 03.

5.4 UC-03: Book Gaming Slot

Step	From	To	Message / Action
1	Customer	Angular Frontend	Open Book a Slot; choose date, station category, station, consecutive slots, and game
2	Angular Frontend	Booking Service	GET /api/bookings/availability (date, station)
3	Booking Service	Booking DB	SELECT: confirmed bookings for station and slots
4	Booking Service	Angular Frontend	Availability, station ratings, and booking summary with charge
5	Customer	Angular Frontend	Proceed to checkout
6	Angular Frontend	Socket.IO Server	Emit lock_slot (station, slot, user)
7	Socket.IO Server	Socket.IO Server	Validate in memory; acquire 5-minute TTL lock
8	Socket.IO Server	All Clients	Broadcast slot_locked – clients show "Holding"
9	Angular Frontend	Booking Service	POST /api/bookings (status PENDING); unique constraint guards the slot

Step	From	To	Message / Action
10	Angular Frontend	Customer	Open Payment Modal (see UC-06)
11	Booking Service	Loyalty Service	Award points after confirmation (circuit breaker)
12	Booking Service	Notification Service	Send confirmation by email and SMS
13	Booking Service	Socket.IO Server	HTTP event bridge: slot_booked_live
14	Socket.IO Server	All Clients	Calendar updates without reload
15	Socket.IO Server	All Clients	On TTL expiry or payment failure, broadcast slot_unlocked

Table 13: Sequence – UC-03 Book Gaming Slot

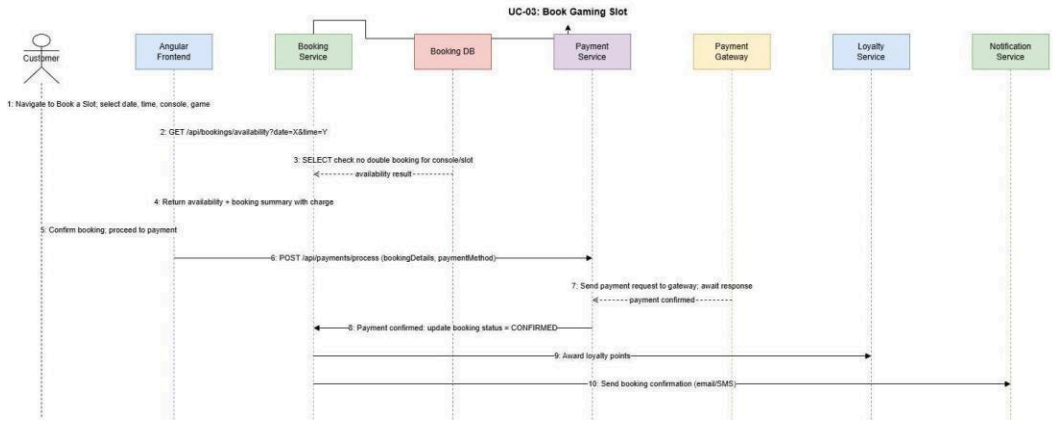


Figure 7: Sequence – UC-03 Book Gaming Slot

Note: diagram shown is from version 02 and is pending update to match version 03.

5.5 UC-04: Cancel / Reschedule Booking

Step	From	To	Message / Action
1	Customer	Angular Frontend	Select booking in My Bookings; choose Cancel or Reschedule
2	Angular Frontend	Booking Service	Cancel or reschedule request via Gateway
3	Booking Service	Booking Service	Verify startTime - now > 2 hours; otherwise return TOO_LATE
4	Booking Service	Booking DB	Cancel: status = CANCELLED. Reschedule: validate new slot within 30 days; update times
5	Booking Service	Loyalty Service	LoyaltyClient.reversePoints(bookingId) (circuit breaker; fallback queues compensation)
6	Loyalty Service	Loyalty DB	Insert REVERSED transaction; decrement balance
7	Booking Service	Payment Service	Initiate refund where applicable
8	Booking Service	Booking DB	On any failed downstream step, insert compensating_transactions row (PENDING) for retry
9	Booking Service	Socket.IO Server	HTTP event bridge: slot_unlocked (and new slot lock for reschedule)

Step	From	To	Message / Action
10	Socket.IO Server	All Clients	Slot becomes selectable on the calendar
11	Booking Service	Notification Service	Notify the customer

Table 14: Sequence – UC-04 Cancel / Reschedule Booking

5.6 UC-05: Admin Book Slot for Walk-in Customer

Step	From	To	Message / Action
1	Admin	Angular Frontend	Navigate to Admin Booking panel; select slot, station, game
2	Angular Frontend	Booking Service	GET /api/admin/bookings/availability
3	Admin	Angular Frontend	Enter walk-in customer name; submit booking
4	Angular Frontend	Booking Service	POST /api/admin/bookings (booking details, type=WALKIN)
5	Booking Service	Booking DB	INSERT: booking record; unique constraint protects the slot
6	Admin	Angular Frontend	Record cash payment: enter amount received

Step	From	To	Message / Action
7	Angular Frontend	Payment Service	POST /api/payments/cash (bookingId, amount)
8	Payment Service	Payment DB	INSERT: payment record (method=CASH, status=COMPLETED)
9	Payment Service	Booking Service	Confirm booking (chained confirmation)
10	Booking Service	Socket.IO Server	slot_booked_live event
11	Payment Service	Angular Frontend	Generate and display printable receipt

Table 15: Sequence – UC-05 Admin Walk-in Booking

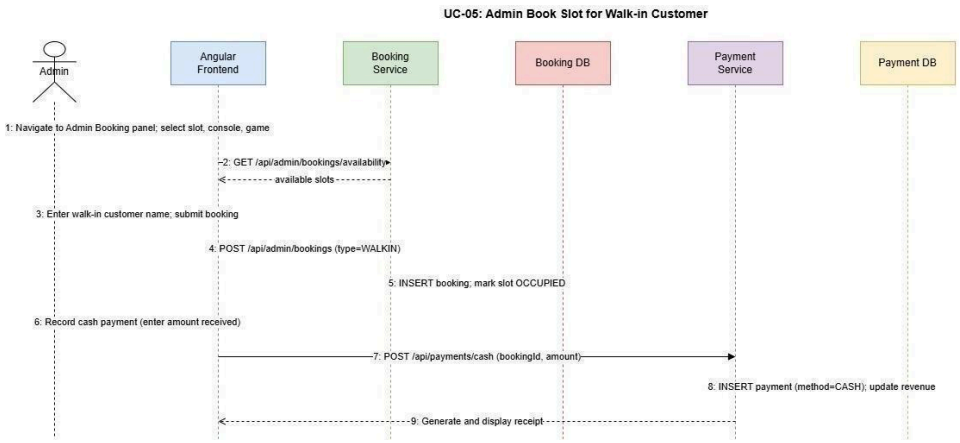


Figure 8: Sequence – UC-05 Admin Walk-in Booking

Note: diagram shown is from version 02 and is pending update to match version 03.

5.7 UC-06: Make Online Payment (PayHere, IPN and Saga)

Step	From	To	Message / Action
1	Customer	Angular Frontend	Enter card details; client formats input and validates with the Luhn algorithm
2	Angular Frontend	Payment Service	Request payment token (bookingId, amount, loyaltyPointsUsed)
3	Payment Service	Payment DB	INSERT payment (status=PENDING, gateway_order_id)
4	Payment Service	Payment Service	Compute PayHere MD5 merchant signature
5	Angular Frontend	PayHere	Customer submits payment
6	PayHere	API Gateway	Asynchronous IPN: POST /api/payments/payhere/notify (JWT-exempt)
7	API Gateway	Payment Service	Forward IPN
8	Payment Service	Payment Service	Validate MD5 signature; reject if invalid
9	Payment Service	Payment DB	Update status to COMPLETED or FAILED

Step	From	To	Message / Action
10	Payment Service	Booking Service	Report result (circuit breaker); Booking Service marks booking CONFIRMED
11	Booking Service	Loyalty Service	Award points; record redemption if used
12	Booking Service	Notification Service	Send email and SMS
13	Booking Service	Socket.IO Server	slot_booked_live event
14	Booking Service	Saga Compensation	On FAILED payment or downstream failure: SagaCompensationService restores points, releases the hold, and requests refund; record in compensating_transactions

Table 16: Sequence – UC-06 Online Payment

5.8 UC-08: Register for Tournament

Step	From	To	Message / Action
1	Customer	Angular Frontend	Navigate to Tournaments page; view available tournaments and ticker banner
2	Angular Frontend	Tournament Service	GET /api/tournaments?status=UPCOMING

Step	From	To	Message / Action
3	Customer	Angular Frontend	Select tournament; click Register
4	Tournament Service	Tournament DB	Check: registration open? participant_count < max_participants?
5	Angular Frontend	Payment Service	Pay entry fee (as UC-06)
6	Payment Service	Tournament Service	Payment confirmed: register participant (store user_name, user_email); increment participant_count
7	Tournament Service	Notification Service	Send confirmation notification to customer
8	Tournament Service	Socket.IO Server	Update ticker (spots remaining)
9	Tournament Service	Notification Service / Socket.IO	On tournament cancellation: email and real-time alert to all participants

Table 17: Sequence – UC-08 Register for Tournament

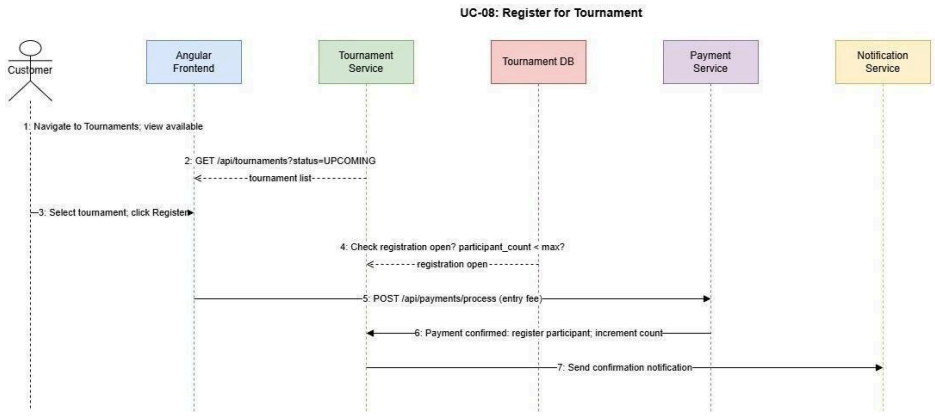


Figure 9: Sequence – UC-08 Register for Tournament

Note: diagram shown is from version 02 and is pending update to match version 03.

5.9 UC-11: Redeem Loyalty Points

Step	From	To	Message / Action
1	Customer	Angular Frontend	Proceed to checkout for a held slot
2	Angular Frontend	Loyalty Service	GET /api/loyalty/balance (userId)
3	Angular Frontend	Customer	Display available points, tier, and equivalent discount
4	Customer	Angular Frontend	Select 'Redeem Points'; view discounted total
5	Angular Frontend	Payment Service	POST payment (amount after discount, loyaltyPointsUsed)
6	Booking Service	Loyalty Service	After confirmation: deduct points; log REDEEMED transaction with balance_after

Step	From	To	Message / Action
7	Booking Service	Saga Compensation	If payment fails or booking is later cancelled: restore points (compensating transaction)
8	Angular Frontend	Customer	Booking confirmed; updated loyalty balance displayed

Table 18: Sequence – UC-11 Redeem Loyalty Points

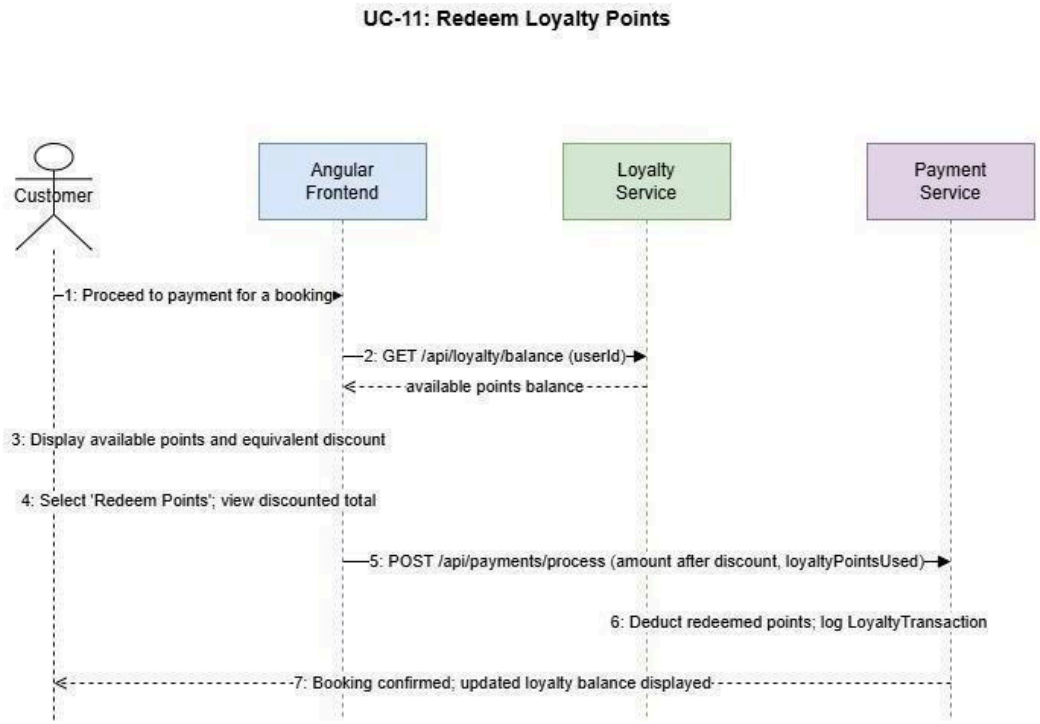


Figure 10: Sequence – UC-11 Redeem Loyalty Points

Note: diagram shown is from version 02 and is pending update to match version 03.

5.10 UC-15: Submit Session Review

Step	From	To	Message / Action
1	Customer	Angular Frontend	Open My Bookings; choose Rate Session on a completed booking
2	Angular Frontend	Customer	Show five-star review modal
3	Customer	Angular Frontend	Select rating (1–5) and optional comment; submit
4	Angular Frontend	Booking Service	POST /api/reviews (bookingId, rating, comment)
5	Booking Service	Booking DB	Verify booking is COMPLETED, owned by the user, and has no review (unique booking_id)
6	Booking Service	Booking DB	INSERT review; recompute station average rating
7	Booking Service	Angular Frontend	201 Created; review appears in My Reviews and updates station badge

Table 19: Sequence – UC-15 Submit Session Review

Chapter 6 UI/UX Design

6.1 Overview

The Gaming Castle web application shall provide a modern, responsive, and accessible user interface. The design follows a dark-themed gaming aesthetic that is consistent with the Gaming Castle brand identity. All pages shall render correctly on desktop, tablet, and mobile devices (minimum viewport width of 320px), in conformance with NFR-07. The frontend is built with Angular 19 standalone components, reactive signals, OnPush change detection, and a custom SCSS design system.

6.2 Design Principles

- Consistency: A unified colour scheme, typography (sans-serif headings, readable body text), and iconography shall be applied across all pages.
- Simplicity: Core customer workflows (slot booking, tournament registration, loyalty redemption) shall be completable in a maximum of 5 minutes by a first-time user.
- Feedback: All form submissions, payment processing stages, and booking confirmations shall provide visible status feedback via toast notifications, inline messages, or real-time badges.
- Accessibility: The interface shall comply with WCAG 2.1 Level AA, ensuring usability for users with visual or motor impairments.
- Real-time awareness: Slot states (available, "Holding", booked) shall update live without a page reload.

6.3 Key Screens and Wireframe Descriptions

6.3.1 Customer – Home Page

The home page shall display a hero section with a "Book a Slot" call-to-action button presenting the three station categories (PS5, PC Esports, VR), a featured tournament card with the live ticker banner, featured games, and a loyalty balance and tier widget for authenticated users. The top navigation bar shall include links to Book a Slot, Tournaments, My Bookings, Loyalty Rewards, and Profile, and the notification bell with an unread badge.

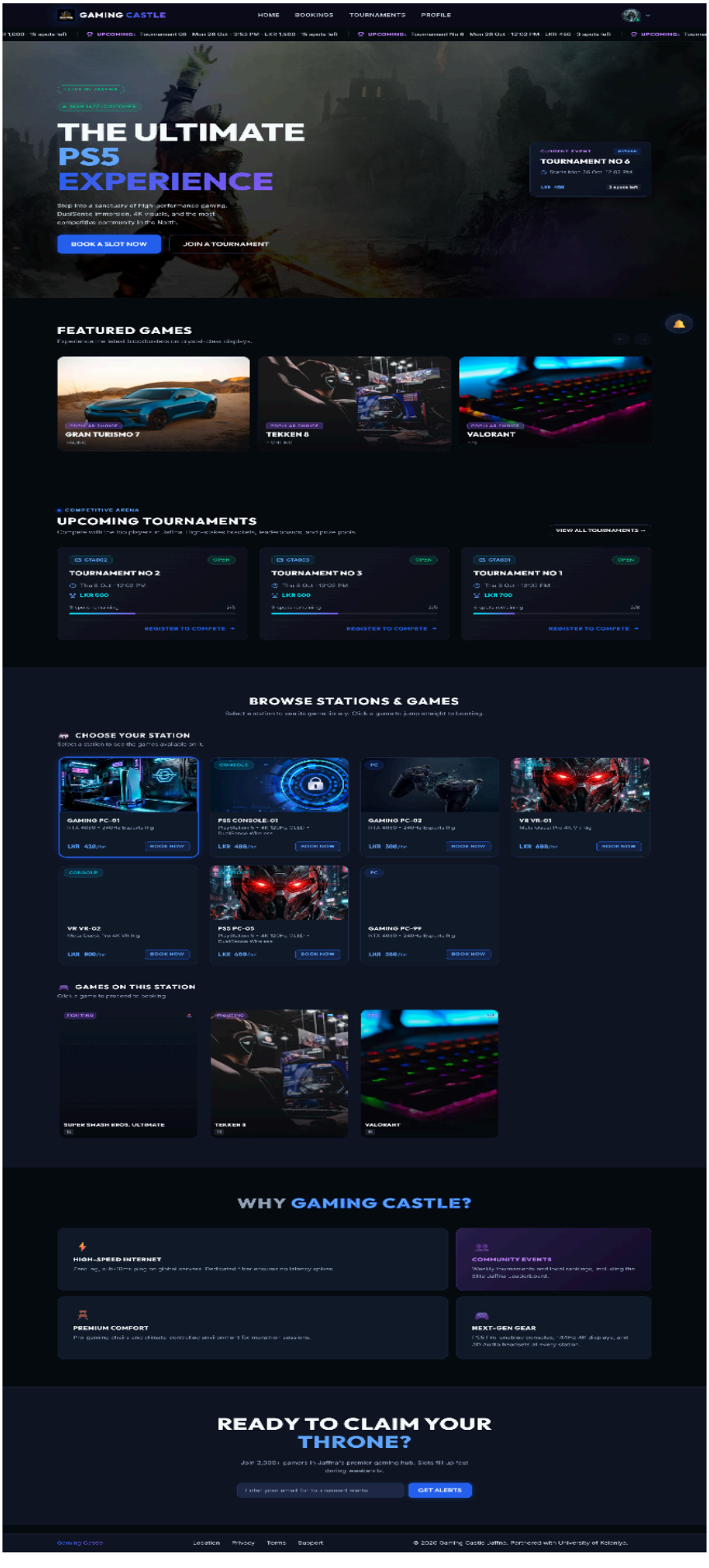

GAMING CASTLE
HOME
BOOKINGS
TOURNAMENTS
PROFILE
THE ULTIMATE
PS5
EXPERIENCE
Step into a sanctuary of high-performance gaming. DualSense immersion, 4K visuals, and the most competitive community in the North.
BOOK A SLOT NOW
JOIN A TOURNAMENT
TOURNAMENT NO 6
FEATURED GAMES
GRAN TURISMO 7
TEKKEN 8
VALORANT
COMPETITIVE ARENA
UPCOMING TOURNAMENTS
VIEW ALL TOURNAMENTS →
TOURNAMENT NO 2
LKR 500
REGISTER TO COMPETE →
TOURNAMENT NO 3
LKR 500
REGISTER TO COMPETE →
TOURNAMENT NO 1
LKR 700
REGISTER TO COMPETE →
BROWSE STATIONS & GAMES
CHOOSE YOUR STATION
GAMING PC-01
LKR 430/hr
BOOK NOW
PS5 CONSOLE-01
LKR 480/hr
BOOK NOW
GAMING PC-02
LKR 300/hr
BOOK NOW
VR VR-01
LKR 680/hr
BOOK NOW
VR VR-02
LKR 800/hr
BOOK NOW
PS5 PC-05
LKR 650/hr
BOOK NOW
GAMING PC-99
LKR 380/hr
BOOK NOW
GAMES ON THIS STATION
SUPER SMASH BROS. ULTIMATE
TEKKEN 8
VALORANT
WHY GAMING CASTLE?
HIGH-SPEED INTERNET
COMMUNITY EVENTS
PREMIUM COMFORT
NEXT-GEN GEAR
READY TO CLAIM YOUR
THRONE?
GET ALERTS
Location
Privacy
Terms
Support

Figure 11: UI/UX Design - Customer – Home Page

6.3.2 Customer – Slot Booking Page

The slot booking page shall present an interactive calendar date picker, a station category selector (PS5, PC, VR), station cards showing image, hourly rate, and average star rating, a time slot grid for consecutive hourly selection, a game selector limited to compatible active games,

and a live booking summary panel with total price. Slot states are shown with colour and text labels: Available, Holding (locked by another customer for up to 5 minutes), and Booked. Held and booked slots shall be visually disabled and not selectable. After the customer proceeds to checkout, a countdown shows the remaining hold time.

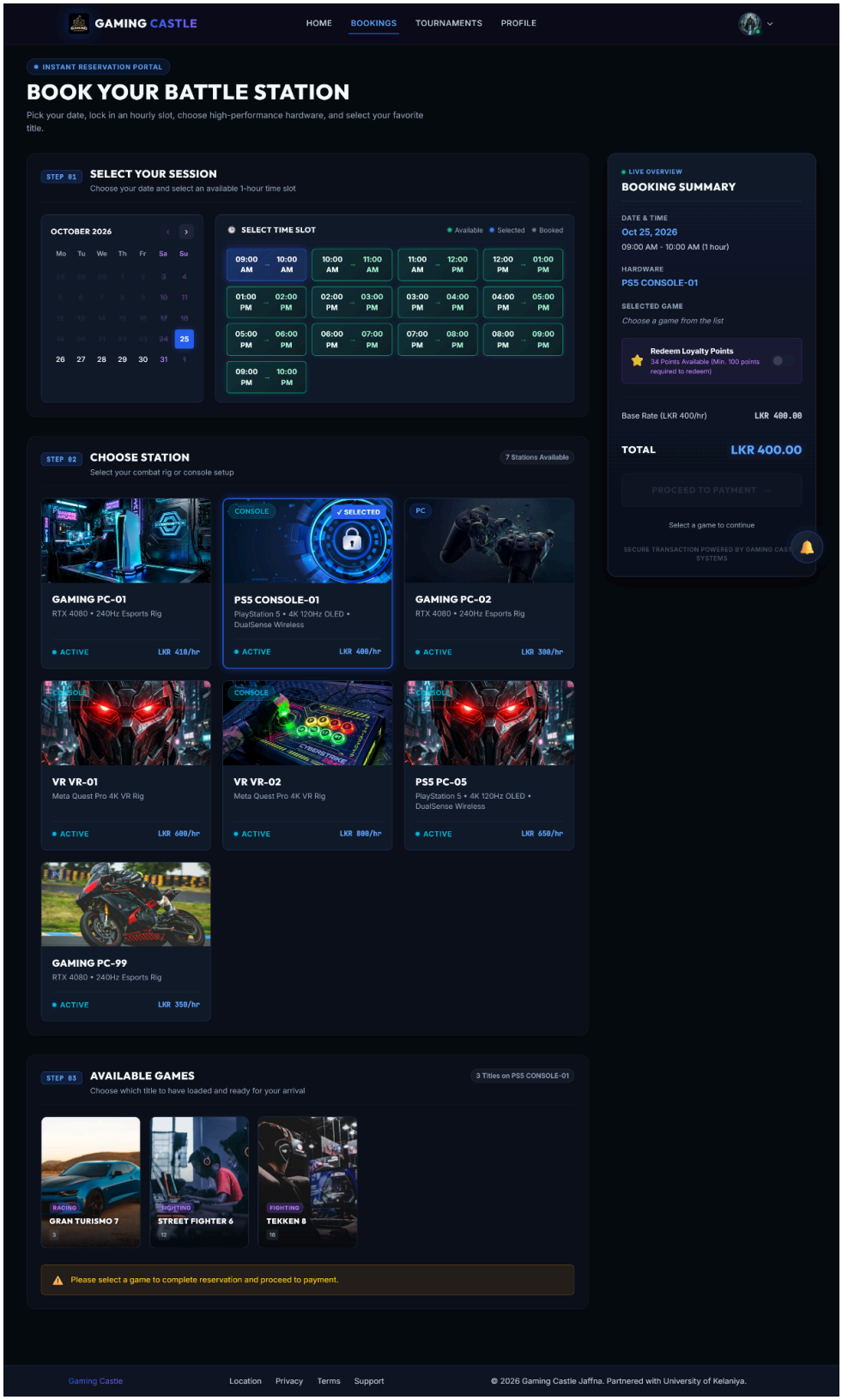


Figure 12: UI/UX Design - Customer – Slot Booking Page

6.3.3 Customer – Payment Page

The payment page shall display the booking summary and total in LKR, a loyalty points redemption toggle showing the available balance and resulting discount, and the PayHere payment method options. The card form auto-formats the number in 4-digit groups and the expiry as MM/YY, shows the detected card brand, and validates the number with the Luhn algorithm before enabling the Pay button. Card data is never stored by Gaming Castle.

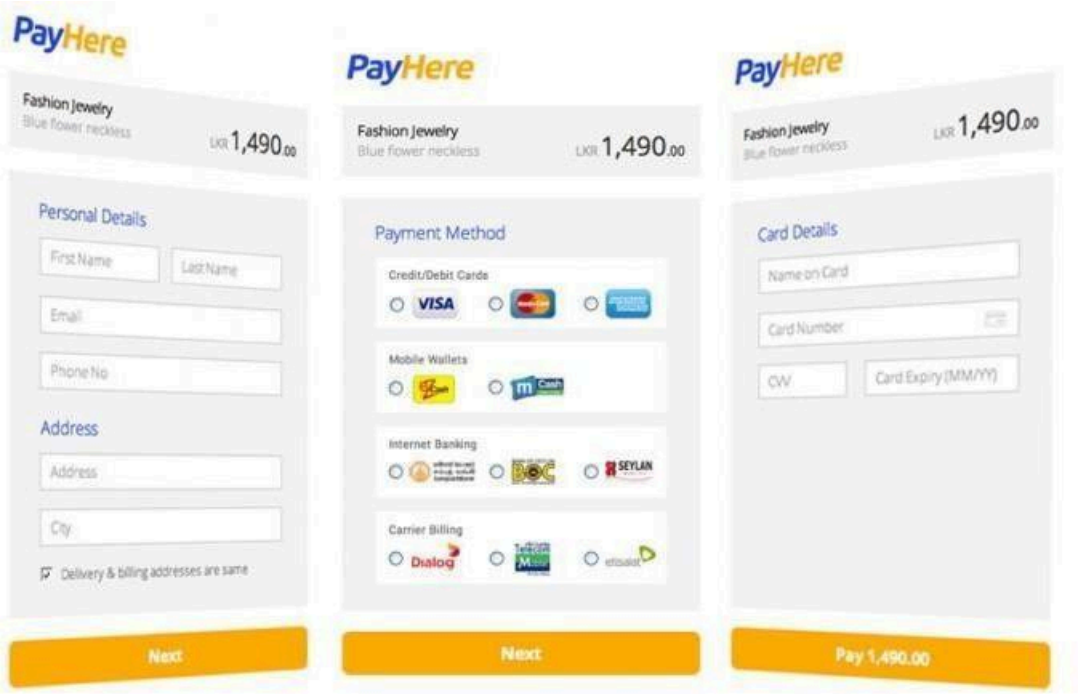


Figure 13: UI/UX Design - Customer – Payment Page

Note: diagram shown is from version 02 and is pending update to match version 03.

6.3.4 Customer – Tournament Page

The tournaments page shall list all available and upcoming tournaments in card format, showing the game title, date, entry fee, prize pool, remaining spots, and a register button, with a ticker banner for the featured event. Closed or full tournaments shall be displayed with a disabled state and a status badge. A registered customer is alerted in real time if a tournament is cancelled.

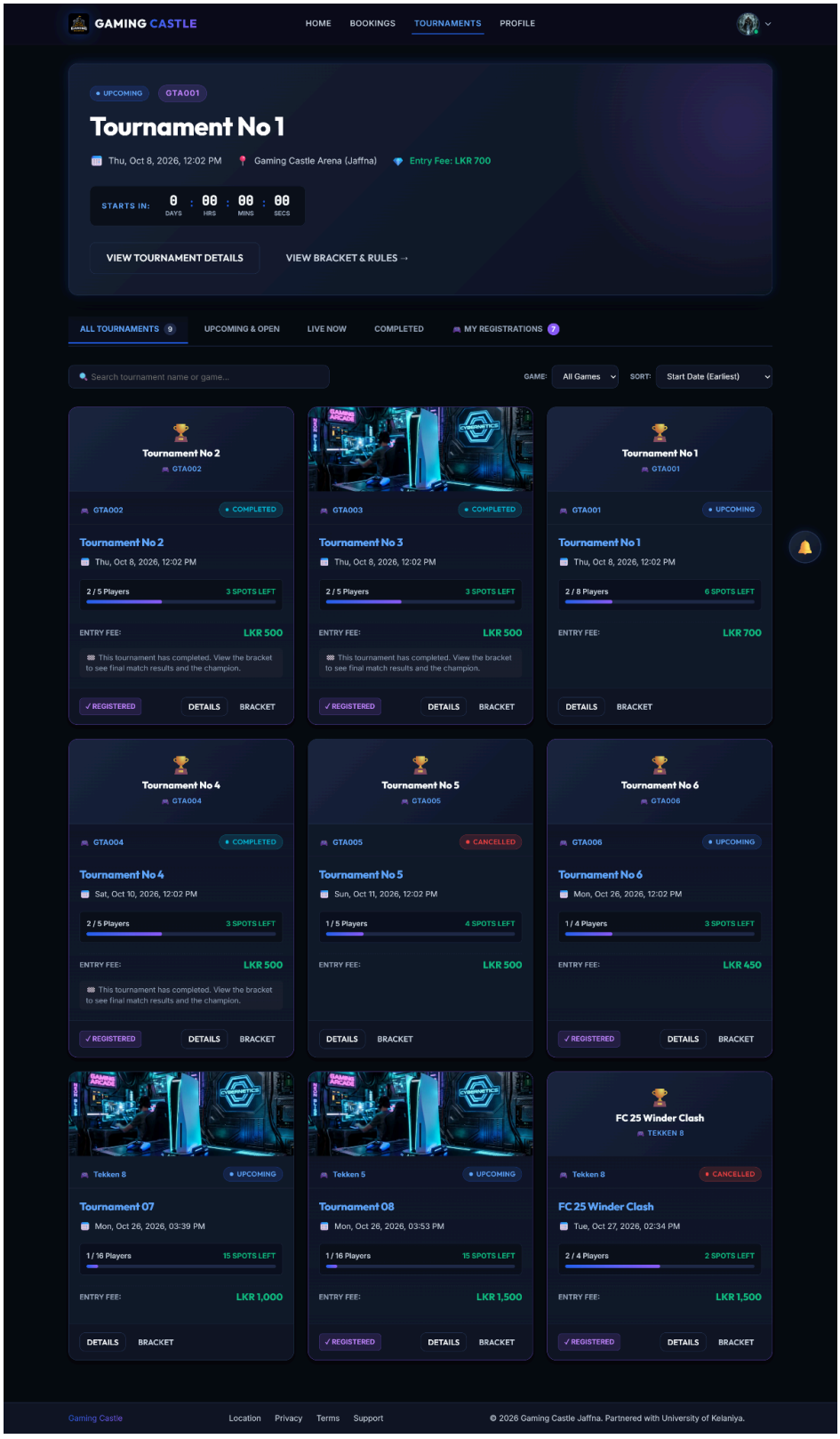


Figure 14: UI/UX Design - Customer – Tournament Page

6.3.5 Customer – My Bookings and My Reviews

My Bookings lists upcoming and past bookings with Cancel and Reschedule actions (disabled with an explanatory message within 2 hours of the start) and a Rate Session action on completed

bookings. The review modal offers an interactive five-star control and an optional comment. My Reviews lists the customer's submitted reviews.

6.3.6 Admin – Dashboard

The admin dashboard shall provide summary widgets for today's revenue, active sessions and station utilisation, upcoming bookings, loyalty points awarded, and the tournament ticker, together with a revenue trend chart and quick actions (record walk-in, create tournament, manage stations and games, edit users). The notification centre plays an audio chime on new events and shows an unread counter. A left-side navigation panel shall provide access to Bookings, Tournaments, Users, Stations and Games, Payments, and Reports sections.

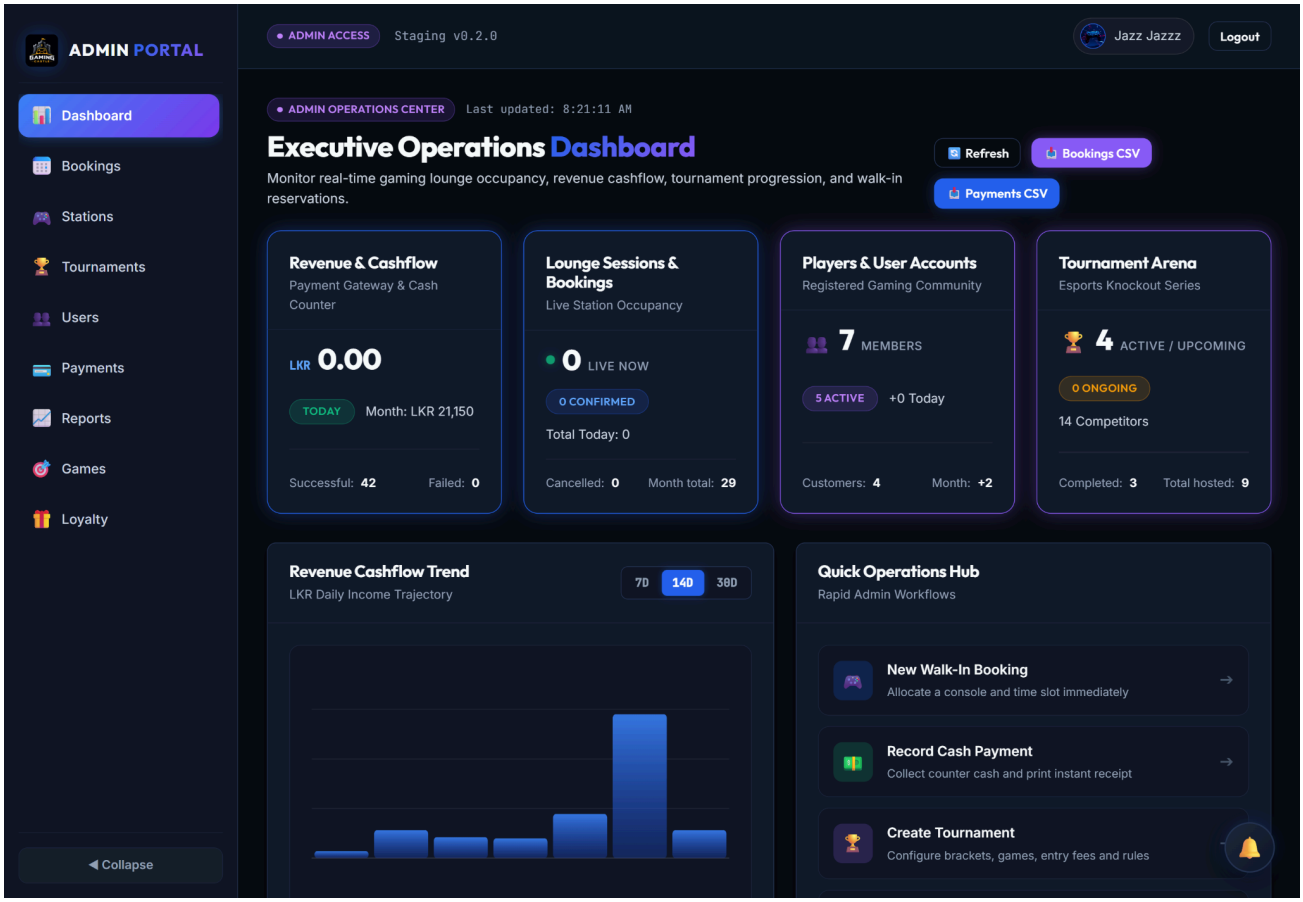


Figure 15: UI/UX Design - Admin – Dashboard

6.3.7 Admin – Booking Management

The admin booking management page shall display a searchable and filterable table of all bookings (date, customer, station, type, payment status) with live occupancy per station, an inline walk-in booking form with cash payment and printable receipt, and action buttons for cancel and reschedule. A booking detail panel shall be accessible on row selection. A station and

game management view edits station type, hourly rate, image, activation, and game assignments.

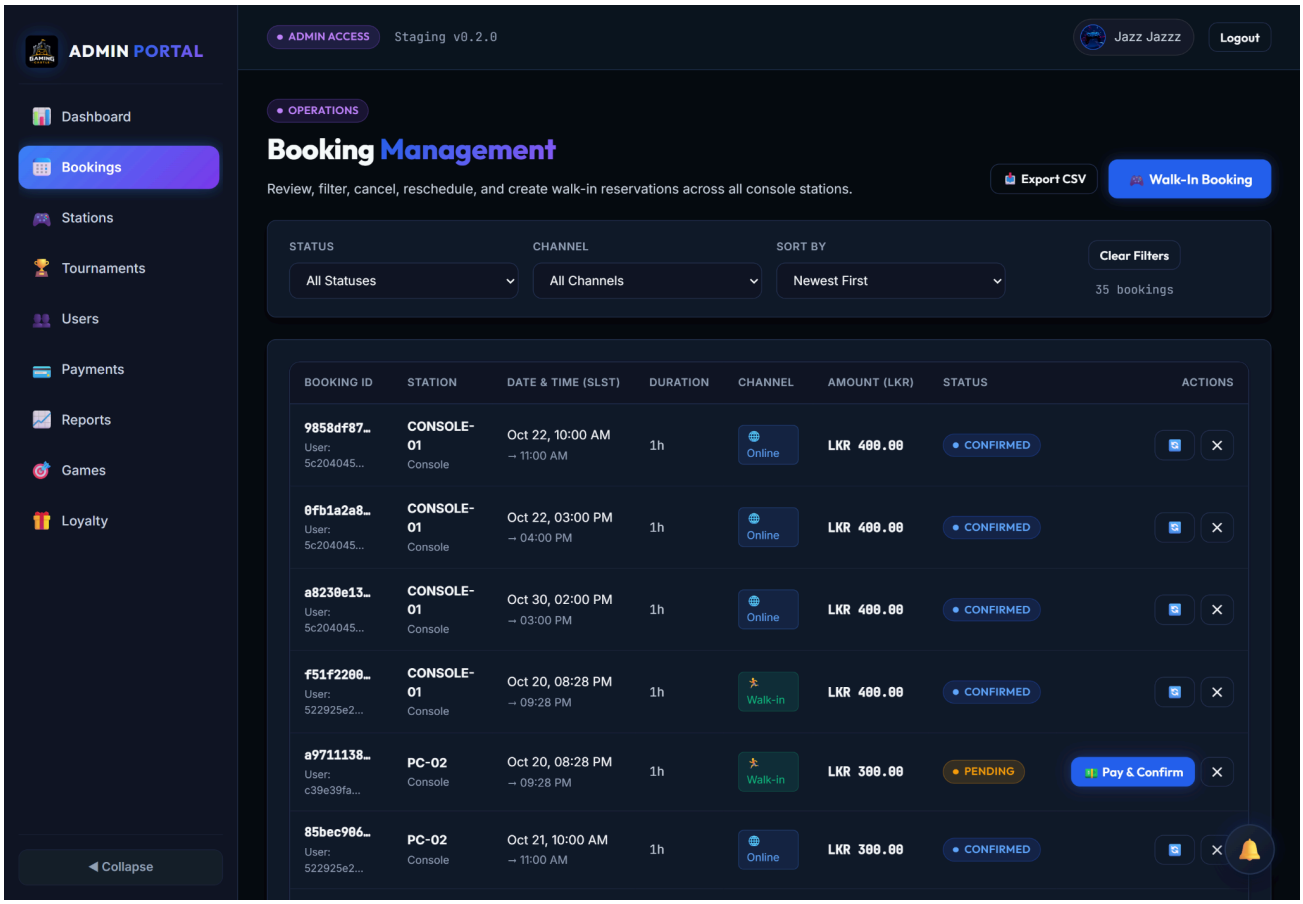


Figure 16: UI/UX Design - Admin – Booking Management

Note: diagram shown is from version 02 and is pending update to match version 03.

6.4 Navigation Flow

User Role	Navigation Path
Customer	Login → Home → Book Slot (select category and station) → Checkout (slot held) → Payment → Confirmation
Customer	Login → My Bookings → Cancel / Reschedule
Customer	Login → My Bookings → Rate Session → My Reviews
Customer	Login → Tournaments → Tournament Detail → Register → Payment
Customer	Login → Profile → Loyalty Rewards → Redeem (on Payment page)
Admin	Login → Admin Dashboard → Manage Bookings / Tournaments / Users / Stations and Games / Reports
Customer	Login → Logout → Home
Customer	Login → Forgot Password → Enter Email or Phone → Enter OTP Code → Set New Password → Login

Table 20: Navigation Flow by Role

6.5 Accessibility Considerations

The following WCAG 2.1 Level AA conformance requirements shall be implemented:

- All form inputs shall include associated label elements and ARIA attributes for screen reader compatibility.

- Colour contrast ratios shall meet the minimum of 4.5:1 for normal text and 3:1 for large text.
- All interactive elements (buttons, links, form fields, the review star control, and the card form) shall be keyboard-navigable and shall display a visible focus indicator.
- Error messages shall be communicated both visually and via ARIA live regions so that screen readers announce validation failures; live slot changes shall be announced politely.
- All images and icons used decoratively shall have an empty alt attribute; informational images shall include descriptive alt text.
- The system shall not rely solely on colour to convey information; status indicators shall include text labels (e.g., Available, Holding, Booked).

Chapter 7 Security Design

7.1 Authentication Mechanism

Authentication shall be implemented using JSON Web Tokens (JWT). Upon successful login, the User Service shall generate a signed JWT containing the user's ID, role, and expiration timestamp. The token shall be transmitted to the client and stored in a secure HTTP-only cookie to prevent XSS-based token theft.

- The cookie shall be configured with the attributes HttpOnly, Secure, and SameSite=Strict.
- Token expiry shall be set to 60 minutes for customer sessions and 30 minutes for admin sessions.
- A refresh token mechanism shall be implemented to allow seamless session renewal without requiring the user to log in again. The refresh token shall also be stored in a secure HTTP-only cookie.
- For each request, the JWT shall be automatically included via cookies and validated by the API Gateway before forwarding the request to the appropriate microservice.
- The PayHere IPN endpoint is the only business endpoint exempt from JWT validation and is protected by signature validation instead (see 7.3).

7.2 Authorisation Levels

Role-Based Access Control (RBAC) shall be enforced across all system endpoints. Two roles are defined: CUSTOMER and ADMIN; the PayHere IPN call is treated as a system caller.

Resource / Endpoint	Guest	CUSTOMER	ADMIN	System (IPN)
Register / Login	✓	✓	✓	✗
View station availability and ratings	✓	✓	✓	✗
Book Gaming Slot / Hold Slot	✗	✓	✗	✗
Cancel / Reschedule Own Booking	✗	✓	✗	✗
Submit / View Own Reviews	✗	✓	✗	✗
View All Reviews	✗	✗	✓	✗
Admin Walk-in Booking and Cash Payment	✗	✗	✓	✗
Register for Tournament	✗	✓	✗	✗
Manage Tournament	✗	✗	✓	✗
View Own Loyalty Points	✗	✓	✗	✗

Resource / Endpoint	Guest	CUSTOMER	ADMIN	System (IPN)
Manage Stations and Games	✗	✗	✓	✗
View Admin Dashboard / Generate Reports	✗	✗	✓	✗
Manage User Accounts	✗	✗	✓	✗
POST /api/payments/payhere/notify	✗	✗	✗	✓ (signature required)

Table 21: Role-Based Access Control Matrix

7.3 Data Protection Approach

- All customer passwords shall be stored exclusively as bcrypt hashes with a minimum cost factor of 12. Plain-text password storage is strictly prohibited.
- Personally identifiable information (PII) including names, email addresses, and phone numbers shall be stored in compliance with applicable data protection regulations.
- Payment card data shall never be persisted in any Gaming Castle database; all card processing shall be delegated entirely to PayHere, which is PCI DSS compliant. The client-side Luhn check is a usability control only.
- The PayHere IPN webhook is whitelisted from JWT authentication, but each notification is accepted only if its MD5 signature matches the value computed with the merchant secret; handling is idempotent so repeated notifications do not duplicate effects.
- Database access credentials, JWT secrets, and PayHere merchant secrets shall be stored as environment variables and shall not be hardcoded in source code or committed to version control.

- Each service shall use only its own database credentials (database-per-service isolation).

7.4 Input Validation and Sanitisation Strategy

- All user inputs shall be validated on the client side (Angular reactive forms) and on the server side (Spring Boot Bean Validation) independently. Client-side validation alone shall not be relied upon as a security measure.
- All string inputs shall be sanitised to prevent Cross-Site Scripting (XSS) attacks. Angular's built-in DOM sanitiser shall be used on the frontend, and review comments and other free text shall be escaped on output.
- All database queries shall use parameterised queries or JPA repositories to prevent SQL injection attacks.
- Avatar and tournament image uploads shall be restricted to whitelisted image MIME types and maximum file size limits.
- The system shall undergo an OWASP Top 10 vulnerability assessment prior to production deployment, in conformance with NFR-12.

7.5 Rate Limiting and Abuse Protection

- The AuthRateLimitFilter token bucket (capacity 10, refill 5 tokens per second per client IP) protects /api/auth/login, /api/auth/register, and /api/auth/password/** against brute-force and credential-stuffing attacks; excess requests receive HTTP 429 with Retry-After.
- Rate-limit state is held in gateway memory; if the gateway is scaled to several instances, limits apply per instance unless a shared store is introduced.

7.6 Encryption Standards

Context	Standard	Notes
Data in Transit	TLS 1.2 / 1.3	Enforced for all HTTP and WebSocket (wss) communications (HTTPS mandatory)

Password Storage	bcrypt (cost factor 12)	No plain-text or MD5/SHA-1 password hashing permitted

Context	Standard	Notes
JWT Signing	HMAC-SHA256 (HS256)	Secret key stored in environment variable; RS256 optional for distributed scenarios
Database at Rest	AES-256	Cloud provider managed encryption for the PostgreSQL volumes
Session Tokens	HTTP-only Secure Cookies	Stored in HTTP-only cookies; not accessible to JavaScript
Payment Signature	MD5 (PayHere scheme)	Used only because PayHere mandates it for request and IPN signatures; never used for passwords or tokens

Table 22: Encryption Standards

7.7 Account Security

- Brute-force protection: accounts shall be temporarily locked (lockout_until) after 5 consecutive failed login attempts, in addition to gateway rate limiting, as described in the Login sequence diagram.
- Password reset and account reactivation shall require a time-limited one-time code sent to the registered email address or phone number; repeated failed code attempts are locked out.
- Admin accounts shall be created exclusively by existing administrators; self-registration for the Admin role shall not be permitted.

Chapter 8 Deployment Design

8.1 Hosting Plan

The Gaming Castle system shall be deployed on Amazon Web Services (AWS) using AWS Free Tier for initial academic deployment and scalable paid tiers for production. The following AWS services shall be utilised:

Component	AWS Service	Purpose
Frontend	AWS S3 + CloudFront	Angular 19 static build served via CDN for low latency
API Gateway and Eureka	AWS ECS Fargate	Containers for Spring Cloud Gateway and the service registry
Microservices	AWS ECS Fargate	One container service for each of the six Spring Boot services
Real-Time Server	AWS ECS Fargate	Node.js Socket.IO container (single instance while locks are in memory)
Database	AWS RDS (PostgreSQL 16)	One database per service, with automated backups
Load Balancer	AWS ALB	HTTPS and WebSocket traffic distribution
Container Registry	AWS ECR	Stores Docker images for each component

Table 23: Hosting Plan

8.2 Hardware Requirements

The system is designed for cloud-based deployment and does not require dedicated physical hardware. The recommended minimum specifications per containerised service instance are:

- vCPU: 0.5 vCPU per instance (scalable to 2 vCPU under load)
- Memory: 512 MB RAM per instance (scalable to 2 GB under load)
- Storage: 50 GB SSD for the RDS PostgreSQL instance (shared across the service databases or per instance, as provisioned)
- Network: Stable internet connectivity at the Gaming Castle admin terminal (minimum 10 Mbps)

8.3 Environment Specification

Environment	Infrastructure	Purpose
Development	Local machine; Docker Compose with PostgreSQL 16 containers, Eureka, gateway, services, and Socket.IO server	Individual feature development and unit testing
Staging	AWS ECS Fargate (reduced capacity); PayHere sandbox	Integration testing, UAT, and pre-release validation
Production	AWS ECS Fargate (full capacity) + RDS; PayHere live	Live system accessible by Gaming Castle clients

Table 24: Environment Specification

8.4 CI/CD Pipeline Overview

A Continuous Integration / Continuous Deployment (CI/CD) pipeline shall be implemented using GitHub Actions to automate the build, test, and deployment process. The pipeline stages are as follows:

Stage	Tool	Action
1	GitHub Actions	Trigger on push to main or pull request to main branch
2	Maven / npm	Build backend microservices (mvn package) and frontend (ng build)
3	JUnit, Mockito / frontend test runner	Execute backend unit and integration tests (including Saga and circuit-breaker scenarios) and the 695 frontend unit tests in headless Chrome; fail pipeline on test failure
4	Docker	Build Docker image for each modified service and the real-time server; tag with commit SHA
5	AWS ECR	Push tagged Docker images to AWS Elastic Container Registry
6	AWS ECS	Deploy updated container images to ECS Fargate (staging auto-deploy; production manual approval)
7	GitHub Actions	Notify team of deployment status via commit status and optional Slack alert

Table 25: CI/CD Pipeline Stages

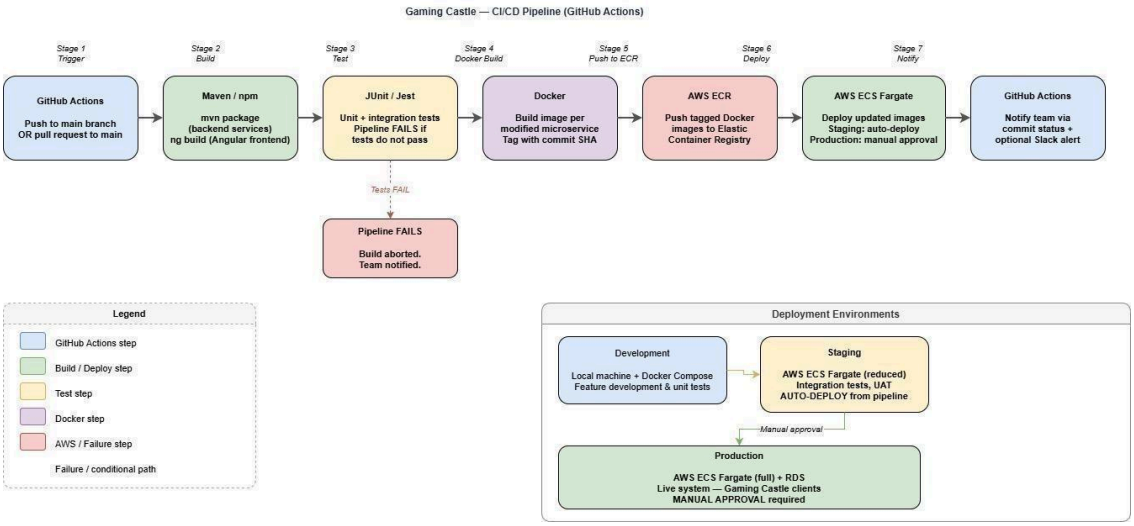


Figure 17: CI/CD Pipeline Diagram

Note: diagram shown is from version 02 and is pending update to match version 03.

8.5 Network Design

The network architecture shall follow AWS best practices for security and availability:

- The system shall be deployed within an AWS Virtual Private Cloud (VPC) divided into public and private subnets.
- The Angular frontend (S3/CloudFront) and the Application Load Balancer shall reside in the public subnet and be accessible from the internet.
- All Spring Boot service containers, the Socket.IO server, Eureka, and the RDS PostgreSQL databases shall reside in private subnets and shall not be directly accessible from the internet.
- Traffic from the internet shall be routed through CloudFront (for the frontend) and the ALB (for API traffic), which forwards requests to the API Gateway container in the private subnet. WebSocket connections to the Socket.IO server pass through the ALB, using WebSocket transport or sticky sessions so that Socket.IO connections stay on one target.
- The PayHere IPN reaches the system through the ALB and gateway only; outbound calls to PayHere, SMTP, and Twilio leave through a NAT gateway.
- Security Groups shall enforce least-privilege ingress and egress rules: only HTTPS (443) shall be exposed publicly; inter-service and database traffic shall be restricted to the internal VPC CIDR range, and each service may reach only its own database.

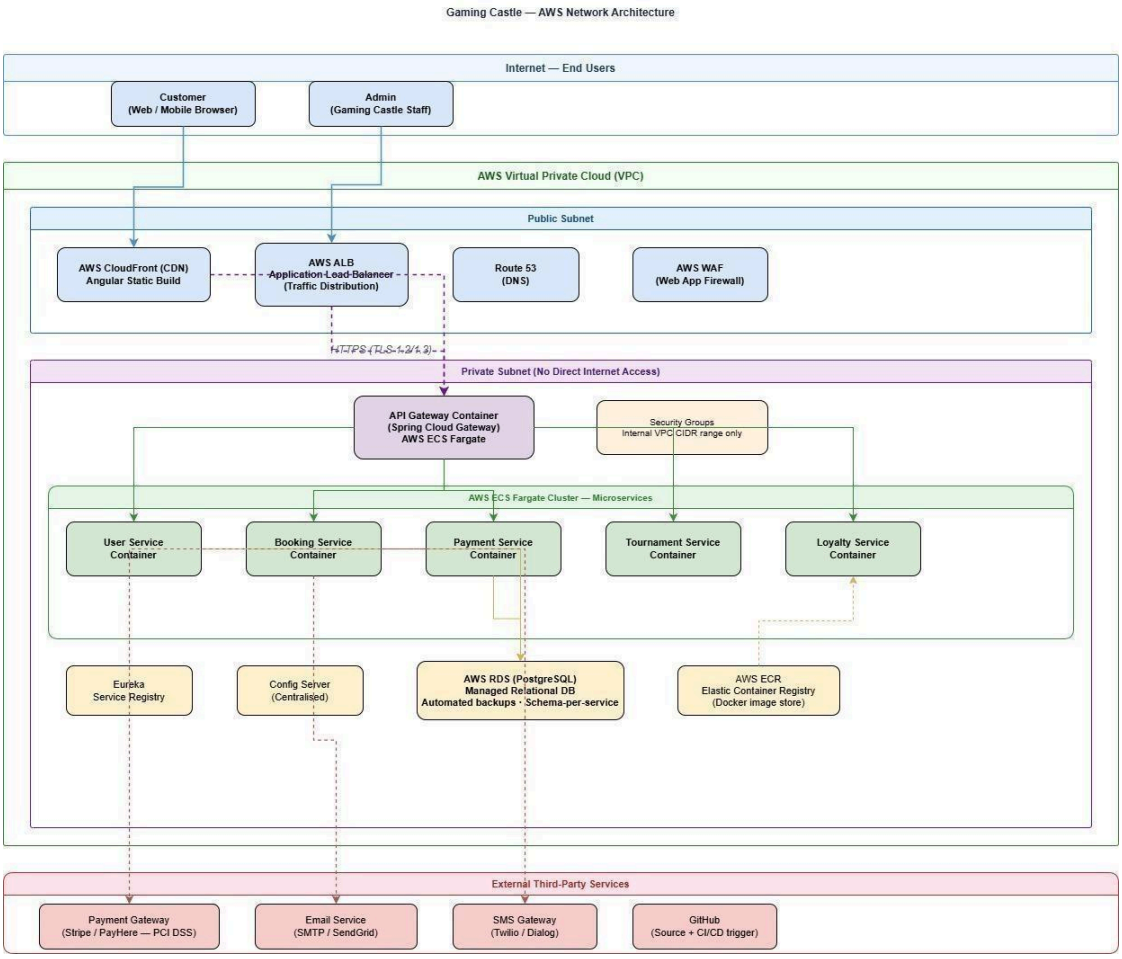


Figure 18: Network Architecture Diagram

Note: diagram shown is from version 02 and is pending update to match version 03.

Chapter 9 References

[1] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.

[2] ISO/IEC, "Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE)," ISO/IEC 25010:2011, 2011.

[3] OWASP Foundation, "OWASP Top Ten," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/. (Accessed Apr. 10, 2026).

[4] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/. (Accessed Apr. 10, 2026).

[5] Spring, "Spring Boot Reference Documentation," Pivotal Software, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/. (Accessed Apr. 10, 2026).

[6] Docker Inc., "Docker Documentation," 2024. [Online]. Available: https://docs.docker.com/. (Accessed Apr. 10, 2026).

[7] Amazon Web Services, "AWS Architecture Center," 2024. [Online]. Available: https://aws.amazon.com/architecture/. (Accessed Apr. 10, 2026).

[8] M. Fowler, "Microservices," martinfowler.com, 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html. (Accessed Apr. 10, 2026).

[9] C. Richardson, *Microservices Patterns*. Manning Publications, 2018.

[10] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 1987.

[11] Resilience4j, "Resilience4j Documentation," [Online]. Available: https://resilience4j.readme.io/. (access date to be added by the authors).

[12] Socket.IO, "Socket.IO Documentation," [Online]. Available: https://socket.io/docs/. (access date to be added by the authors).

[13] PayHere, "PayHere Developer Documentation," [Online]. Available: https://support.payhere.lk/. (access date to be added by the authors).